

HẠ TẦNG KHÓA CÔNG KHAI (PKI)

Báo cáo môn học: Nhập môn An toàn thông tin

Nhóm 8 – Thành viên thực hiện:

Đỗ Trường Giang (20235066) Phạm Gia Hưng (20230036)
Hoàng Minh Khôi (20230042) Nguyễn Thanh Tú (20230075)

Trường Công nghệ Thông tin và Truyền thông (SOICT)
Đại học Bách khoa Hà Nội

Hà Nội, Tháng 6 Năm 2026

Nội dung chương trình

Phần I: Kiến thức nền tảng của PKI (Trọng tâm)

- Đặt vấn đề & Tiến hóa mật mã
- Các thực thể PKI (RootCA, SubCA, RA, VA)
- TLS, OCSP, OCSP Stapling & Trusted Path
- Cấu trúc chứng chỉ X.509 v3
- Kịch bản áp dụng Extensions của X.509 v3

Phần II: Thực nghiệm & Nguồn mở

- Khảo sát hệ sinh thái mã nguồn mở
- Triển khai thử EJBCA trên Docker
- Tương tác EJBCA bằng ngôn ngữ Python
- Thách thức (PQC, Zero Trust)

Phần I: Kiến thức nền tảng của PKI

Đặt vấn đề: Mật mã bất đối xứng và Tấn công MITM

Hạn chế của mật mã bất đối xứng:

- Giải quyết tốt việc trao đổi khóa và ký số không cần gặp trước.
- Tuy nhiên, thiếu cơ chế **ràng buộc danh tính** (identity binding) với khóa công khai.

Tấn công Kẻ đứng giữa (Man-in-the-Middle - MITM):

- Kẻ tấn công (Mallory) đánh chặn khóa công khai thật của Bob.
- Thay thế bằng khóa công khai giả mạo của mình.
- Alice mã hóa dữ liệu bằng khóa giả của Mallory → Bị lộ thông tin.



Figure: Tấn công trung gian khi phân phối khóa thô

Giải pháp: Neo tín nhiệm qua CA

Vai trò của Tổ chức Chứng thực (CA):

- Hoạt động như bên thứ ba tin cậy (Trusted Third Party - TTP).
- Đóng vai trò là **Neo tín nhiệm (Trust Anchor)**.
- Xác minh danh tính và dùng khóa riêng của CA để ký xác thực cho khóa công khai của chủ thể.

Kết quả:

- Client chỉ cần tin tưởng vào khóa công khai của Root CA (nằm trong Trust Store).
- Client tự động xác minh được mọi khóa công khai của máy chủ bằng cách lần chuỗi chữ ký về Root CA.

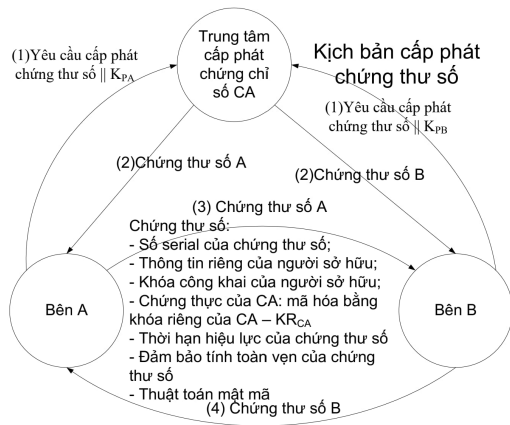


Figure: Mô hình xác thực khóa công khai qua CA

Tiến hóa mật mã: Mật mã đối xứng và hạn chế

Nguyên lý hoạt động

Sử dụng chung khóa bí mật K : $C = E(K, P)$ và $P = D(K, C)$. Thuật toán tiêu biểu: **AES, DES, 3DES**.

Ưu điểm vượt trội:

- Tốc độ xử lý cực nhanh trên phần cứng.
- Đáp ứng truyền tải dữ liệu lớn thời gian thực (video streaming, mã hóa đĩa).

Hạn chế chí mạng:

- **Phân phối khóa:** Catch-22 (cần kênh bảo mật để gửi khóa bí mật).
- **Độ phức tạp khóa:** Mạng n node cần $\frac{n(n-1)}{2} = O(n^2)$ khóa.
- **KDC (Kerberos) giới hạn:** Điểm lỗi tập trung (SPOF), tắc nghẽn hiệu năng, God-mode DB.

Mật mã bất đối xứng và Hàm cửa sập

Nguyên lý Cặp khóa (Khóa công khai PU, Khóa riêng PR):

- **Bảo mật (Secrecy):** Mã hóa bằng *PU* người nhận $\rightarrow$ Chỉ *PR* người nhận giải được.
- **Xác thực/Ký số:** Ký bằng *PR* người gửi $\rightarrow$ Ai cũng giải được bằng *PU* người gửi để xác nhận nguồn gốc.
- **Kết hợp:** Ký bằng *PR* người gửi, mã hóa bằng *PU* người nhận.

Toán học: Hàm một chiều có cửa sập (Trapdoor One-way):

- Tính xuôi $Y = f(X)$ dễ dàng.
- Tính ngược $X = f^{-1}(Y)$ là bất khả thi trừ khi biết khóa riêng *PR* đóng vai trò "cửa sập".

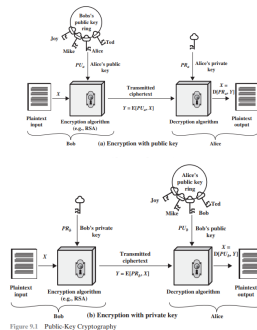


Figure: Mô hình mã hóa (trên) và ký số (dưới)

Thuật toán RSA và Trao đổi khóa Diffie-Hellman

Thuật toán RSA (Phân tích thừa số nguyên tố)

- $n = p \times q$ (p, q là số nguyên tố lớn).
- Khóa công khai $PU = \{e, n\}$, Khóa bí mật $PR = \{d, n\}$.
- Mã hóa: $C = M^e \bmod n$.
- Giải mã: $M = C^d \bmod n$.
- **An toàn:** Chống Timing attack bằng *Blinding* (nhân bản mã với r^e ngẫu nhiên), chống CCA bằng *đệm OAEP*.

Trao đổi khóa Diffie-Hellman

- Dựa trên **Logarit rời rạc** trên trường modulo số nguyên tố lớn q .
- Alice tính: $Y_A = \alpha^{X_A} \bmod q$.
- Bob tính: $Y_B = \alpha^{X_B} \bmod q$.
- Khóa chung: $K = (Y_B)^{X_A} \bmod q = (Y_A)^{X_B} \bmod q = \alpha^{X_A \cdot X_B} \bmod q$.
- **Lỗ hổng:** Không xác thực danh tính $\rightarrow$ dễ bị can thiệp và trao đổi khóa (MITM).

Chữ ký số và Chống chối cãi

Giải quyết điểm yếu của Mật mã đối xứng:

- Khóa đối xứng dùng chung có rủi ro người nhận tự sửa đổi nội dung giao dịch rồi đổ lỗi hoặc người gửi chối bỏ trách nhiệm gửi.

Nguyên lý hoạt động:

- Ký số = Tính Hash của thông điệp + Mã hóa Hash bằng khóa riêng (PR) của người gửi.
- Đạt được **Tính chống chối cãi (Non-repudiation)**: Chỉ có người sở hữu khóa riêng mới tạo được chữ ký hợp lệ.

Các cấp độ giả mạo chữ ký:

- Total break (lộ khóa PR).
- Universal / Selective / Existential forgery.

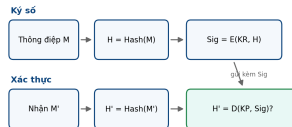


Figure: Sơ đồ tạo và xác thực chữ ký số

Các thực thể PKI: RootCA, SubCA, RA, VA

Các thực thể trong hệ thống PKI

1. Root CA (Tổ chức phát hành chứng chỉ gốc)

- Neo tín nhiệm tối cao của toàn hệ thống.
- Sử dụng **chứng chỉ tự ký (self-signed)**.
- Cần được cách ly vật lý hoàn toàn (*offline, air-gapped*) để bảo mật tuyệt đối khóa riêng.

2. Sub CA / Intermediate CA (CA trung gian)

- Được Root CA ký ủy quyền.
- Chịu trách nhiệm trực tiếp ký và cấp phát chứng chỉ cho thực thể cuối (*leaf certificate*).

3. RA – Registration Authority (Tổ chức đăng ký)

- Xác minh danh tính thực thể của đối tượng yêu cầu.
- Duyệt/Từ chối CSR, nhưng **không trực tiếp ký phát hành** chứng chỉ.

4. VA – Validation Authority (Tổ chức kiểm duyệt)

- Xác thực trạng thái chứng chỉ (còn hiệu lực, bị thu hồi, hay tạm dừng).
- Triển khai qua dịch vụ thư mục công cộng, máy chủ CRL hoặc OCSP Responder.

Mối quan hệ và luồng vận hành của các thực thể

Phân quyền trách nhiệm (Separation of Duty):

- **RA** lo khâu xác thực ngoài đời thực (giấy tờ, tên miền). Giúp CA tránh tiếp xúc trực tiếp mạng Internet.
- **CA** lo khâu ký số mật mã trong phân vùng an toàn cao (HSM).
- **VA** lo khâu phản hồi thông tin kiểm tra cho bên thứ ba nhanh chóng.

Mối quan hệ tương tác:

- Root CA ký cho Sub CA → Sub CA ký cho Leaf Cert.
- Client nộp yêu cầu qua RA → RA duyệt → Sub CA ký cert → Sub CA đồng bộ thông tin thu hồi sang VA → Verifier truy vấn VA để kiểm duyệt cert của Client.

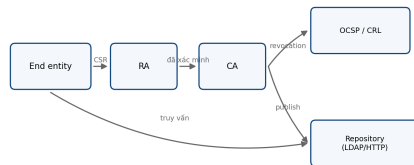


Figure: Quan hệ tương tác CA, RA, VA (Repository)

Quy trình giao tiếp: Client xin cấp chứng chỉ từ RA

Các bước giao tiếp mật mã (CSR Flow)

- ➊ **Khởi tạo cục bộ:** Client tự tạo cặp khóa bí mật/công khai của mình trên máy.
- ➋ **Tạo CSR (PKCS#10):** Client đóng gói thông tin định danh mong muốn + Khóa công khai, dùng khóa riêng ký lên gói này để chứng minh quyền sở hữu khóa (*Proof-of-Possession*).
- ➌ **Gửi yêu cầu:** Client gửi CSR cùng các hồ sơ chứng minh danh tính pháp nhân đến **RA**.
- ➍ **RA Xác minh:** RA kiểm tra tính hợp lệ của CSR, thực hiện các thử thách kiểm soát tên miền (DNS/HTTP challenge) hoặc xác minh giấy tờ của Client.
- ➎ **Chuyển tiếp ký:** Sau khi xác minh thành công, RA ký phê duyệt và gửi yêu cầu đến **CA**.
- ➏ **Ký và Trả kết quả:** CA ký số tạo chứng chỉ X.509, lưu trữ vào Repository, và gửi lại cho Client sử dụng.

Cách VA xử lý yêu cầu kiểm duyệt trạng thái chứng chỉ

Khi Verifier muốn kiểm tra trạng thái chứng chỉ, nó sẽ truy vấn **VA** (Validation Authority):

Quy trình xử lý của VA (OCSP/CRL)

- **Bước 1: Tiếp nhận:** Nhận hash định danh CA và số serial từ Verifier.
- **Bước 2: Tra cứu:** VA tìm kiếm serial trong danh sách thu hồi (từ CA).
- **Bước 3: Tạo phản hồi:** Đóng gói trạng thái (Good, Revoked, Unknown) và ký số bằng khóa riêng của VA.
- **Bước 4: Trả kết quả:** Gửi phản hồi về cho Verifier.

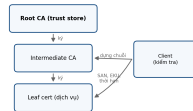


Figure: Kiểm tra chứng chỉ của Verifier

Giao thức mạng: TLS, OCSP, OCSP Stapling & Trusted Path

Giao thức bắt tay TLS (TLS 1.3 Handshake)

TLS 1.3 bảo mật kênh truyền web bằng cách rút ngắn thời gian bắt tay (1-RTT) và ép buộc xác thực chứng chỉ:

- **1. ClientHello:** Client gửi các bộ thuật toán mã hóa hỗ trợ + Key Share đàm phán khóa.
- **2. ServerHello:** Server chọn thuật toán và gửi Key Share của mình.
- **3. Server Certificate:** Server gửi chứng chỉ X.509 của mình kèm chuỗi CA.
- **4. CertificateVerify:** Server dùng khóa riêng của mình ký lên toàn bộ dữ liệu bắt tay trước đó để chứng minh quyền sở hữu khóa.
- **5. Client verify:** Client dựng chuỗi niềm tin, xác minh chữ ký trên chứng chỉ và bắt tay. Kênh mã hóa được thiết lập.

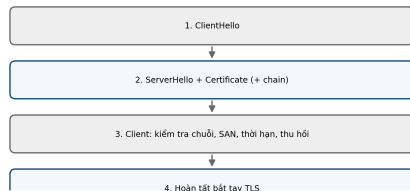


Figure: Sơ đồ bắt tay TLS 1.3

Giao thức OCSP (Online Certificate Status Protocol)

Khái niệm và Nguyên lý (RFC 6960)

OCSP là giao thức thay thế cho CRL để kiểm tra trạng thái thu hồi chứng chỉ theo thời gian thực (real-time) thông qua HTTP.

Quy trình hoạt động:

- Client gửi một truy vấn nhẹ chứa số serial của cert lên máy chủ OCSP của CA.
- OCSP Responder tra cứu cơ sở dữ liệu và trả về kết quả trạng thái chứng chỉ ngay lập tức.

Hạn chế lớn của OCSP truyền thống:

- **Quyền riêng tư:** CA không biết Client đang duyệt web ở trang nào.
- **Hiệu năng:** Client phải tạo thêm một kết nối mạng HTTP phụ đến CA trước khi hoàn thành bắt tay TLS → tăng độ trễ tải trang.

Trạng thái	Ý nghĩa
Good	Chứng chỉ còn hiệu lực và bình thường.
Revoked	Chứng chỉ đã bị thu hồi vĩnh viễn hoặc tạm dừng.
Unknown	VA/CA không có thông tin về số serial này.

Table: 3 trạng thái phản hồi của OCSP

Kỹ thuật OCSP Stapling

Giải pháp tối ưu hóa Hiệu năng và Bảo mật (RFC 6066)

Thay vì Client tự liên hệ VA để hỏi trạng thái cert, **Máy chủ Web (Server)** sẽ đảm nhận việc này.

Quy trình vận hành:

- ➊ Máy chủ Web định kỳ tự kết nối đến máy chủ OCSP của VA để lấy phản hồi trạng thái cert của chính mình.
- ➋ VA gửi phản hồi OCSP đã được ký số và đóng dấu thời gian về cho Máy chủ Web. Máy chủ lưu cache kết quả này.
- ➌ Khi Client bắt tay TLS, Máy chủ Web "đính kèm" (staple) câu trả lời OCSP này gửi kèm chứng chỉ cho Client.

Ưu điểm vượt trội:

- **Tốc độ cực nhanh:** Client không cần kết nối mạng phụ đến VA, giảm 1-RTT.
- **Bảo vệ quyền riêng tư:** VA không hề biết Client đang duyệt web ở trang nào.
- **Đáng tin cậy:** Client vẫn tự xác minh được chữ ký của VA trên gói tin OCSP đính kèm.

Cơ chế Trusted Path (Xác thực đường dẫn chứng chỉ)

Dựng và xác minh chuỗi niềm tin từ chứng chỉ thực thể cuối (*leaf*) về neo tín nhiệm (*Root CA*):

Quy trình kiểm tra đường dẫn (RFC 5280)

- ➊ **Dựng chuỗi (Path Building):** Tìm kiếm các CA phát hành bằng cách khớp AuthorityKeyID và SubjectKeyID.
- ➋ **Xác minh chữ ký:** Dùng khóa công khai của CA cấp trên xác minh chữ ký của CA cấp dưới (đi ngược từ dưới lên).
- ➌ **Kiểm tra ràng buộc:** Hiệu lực thời gian, cờ cA=true (BasicConstraints), và mục đích sử dụng (KeyUsage, EKU).

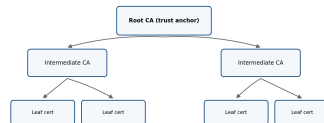


Figure: Chuỗi tin cậy từ Root CA đến Leaf

Chứng chỉ số X.509 v3 & Kịch bản áp dụng Extensions

Cấu trúc chứng chỉ X.509 v3

Định nghĩa bởi ITU-T và chuẩn hóa bởi IETF qua RFC 5280. Cấu trúc logic gồm 3 khối dữ liệu ASN.1 lớn:

1. tbsCertificate (To Be Signed)

Chứa nội dung thông tin cốt lõi của cert:

- Version (v3), SerialNumber (số seri duy nhất).
- Issuer: Tên DN của CA phát hành.
- Validity: Thời hạn (notBefore và notAfter).
- Subject: Tên DN định danh chủ thể được cấp.
- subjectPublicKeyInfo: Khóa công khai và thuật toán.
- Extensions: Các trường mở rộng cấu hình.

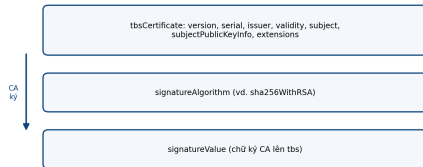


Figure: Cấu trúc logic phân lớp của X.509

Kịch bản áp dụng Extensions X.509 v3 (Phần 1)

Kịch bản 1: HTTPS/TLS Server Auth

Dùng để xác thực danh tính Web Server công cộng.

- **KeyUsage:** `digitalSignature` (xác thực bắt tay), `keyEncipherment` (mã hóa trao đổi khóa RSA) hoặc `keyAgreement` (Diffie-Hellman).
- **ExtendedKeyUsage (EKU):** `serverAuth` (OID: 1.3.6.1.5.5.7.3.1) → Không chế cert chỉ làm Web Server.
- **SubjectAltName (SAN):** Bắt buộc chứa tên miền website (ví dụ: `DNS:www.hust.edu.vn`) để trình duyệt so khớp hostname.

Kịch bản 2: Client Auth (mTLS / VPN)

Xác thực người dùng/thiết bị kết nối hệ thống.

- **KeyUsage:** `digitalSignature`.
- **ExtendedKeyUsage (EKU):** `clientAuth` (OID: 1.3.6.1.5.5.7.3.2).
- **SubjectAltName (SAN):** Chứa email (`rfc822Name`: `giang.dt235066@sis.hust.edu.vn`) hoặc URI nhận diện.

Kịch bản áp dụng Extensions X.509 v3 (Phần 2)

Kịch bản 3: Ký mã nguồn (Code Signing)

Xác minh phần mềm sạch, chống giả mạo driver.

- **KeyUsage:** digitalSignature (ký tệp tin thực thi, exe, dll, dmg).
- **ExtendedKeyUsage (EKU):** codeSigning (OID: 1.3.6.1.5.5.7.3.3)
→ Ngăn chặn kẻ tấn công lấy chứng chỉ HTTPS đi ký cho mã độc.

Kịch bản 4: Chứng chỉ CA (Ủy quyền ký)

Cấp cho các thực thể ký phát hành trong PKI.

- **BasicConstraints:** Cờ cA = True (Bắt buộc). Nếu thiếu hoặc false, client từ chối mọi cert do khóa này ký.
- **KeyUsage:** keyCertSign (quyền ký chứng chỉ) và cRLSign (quyền ký CRL).
- **PathLenConstraint:** Giới hạn cấp CA con tối đa bên dưới.

CSR (PKCS#10) và Họ chuẩn PKCS liên quan

CSR (PKCS#10 - RFC 2986):

- Gói tin chứa thông tin định danh DN, khóa công khai của Client.
- Được ký số bởi chính khóa riêng tương ứng để chứng minh quyền sở hữu khóa bí mật (*Proof-of-Possession*), bảo vệ trước việc mạo danh.

Phân loại theo mức xác thực TLS:

- **DV (Domain Validation):** Tự động xác minh quyền sở hữu tên miền.
- **OV (Organization Validation):** Xác minh pháp lý tổ chức thủ công.
- **EV (Extended Validation):** Xác minh nghiêm ngặt cao nhất.

Họ chuẩn mật mã PKCS phổ biến

- **PKCS#7 (CMS):** Định dạng bao gói dữ liệu mã hóa/ký số (dùng trong chữ ký PDF, S/MIME).
- **PKCS#10:** Cấu trúc yêu cầu ký CSR.
- **PKCS#12 (PFX):** Định dạng chứa đồng thời khóa riêng, chứng chỉ cá nhân và chuỗi CA, được mã hóa bảo vệ bằng mật khẩu mã hóa mạnh ('.p12' / '.pfx').

Ứng dụng PKI trong thực tế

- **HTTPS / TLS 1.3:** Mã hóa kênh truyền web, xác thực máy chủ, tích hợp Certificate Transparency (CT).
- **Ký số văn bản:** Xác thực hóa đơn, văn bản hành chính điện tử. Sử dụng TSA timestamp làm bằng chứng độc lập thời gian.
- **S/MIME:** Ký số và mã hóa email đầu cuối.
- **VPN/IPSec & Code Signing:** Xác thực gateway/client và xác thực phần mềm sạch.
- **eGovernment Việt Nam:** Trục liên thông quốc gia, NEAC Root CA, Cloud CA Remote Signing.

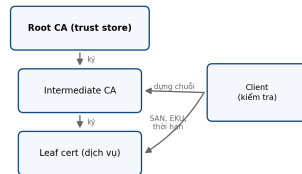


Figure: Bộ quy tắc kiểm tra và dựng chuỗi của Client

Phần II: Thực nghiệm & Hệ thống nguồn mở

Tổng quan Hệ sinh thái PKI Mã nguồn mở

Phân lớp công cụ thực tiễn:

- **Toolkit:** OpenSSL, CFSSL: cryptographic toolkit, tạo khóa, ký CSR. Thiếu quản trị RA và logs.
- **Local GUI:** XCA - quản trị CA trên file SQLite cục bộ (dành cho lab học tập).
- **Enterprise CA Platform:** EJBCA / Dogtag - quản lý profile, tích hợp thiết bị mã hóa cứng HSM (PKCS#11).
- **Internal PKI:** HashiCorp Vault PKI - cấp cert API nội bộ vòng đời siêu ngắn (TTL tính bằng giờ) cho kiến trúc microservices.

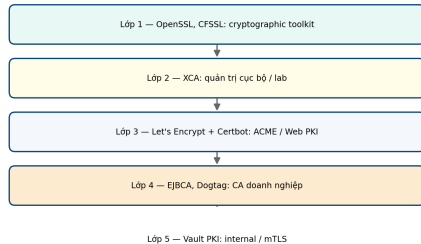


Figure: Phân lớp hệ sinh thái PKI mã nguồn mở

Thực nghiệm: Khởi chạy EJBCA CE trên môi trường Docker

Môi trường Container hóa cho Enterprise CA

EJBCA Enterprise / Community Edition chạy trên nền WildFly (Java) cực kỳ phức tạp để cài đặt trực tiếp. Docker cung cấp môi trường khép kín, sẵn sàng tích hợp:

Lệnh khởi chạy Docker container:

Docker run EJBCA CE

```
docker run -it -d -name ejbca-ce  
-p 80:80 -p 443:443 -p 8443:8443  
-e EJBCA_AUTO_CREATE_ADMIN=true  
keyfactor/ejbca-ce
```

Các cổng dịch vụ hoạt động:

- 80 / 443: Giao diện quản trị, cổng dịch vụ Enroll công cộng.
- 8443: Cổng mTLS yêu cầu client certificate (quản trị viên cao cấp).

Kiến trúc triển khai:

- **Cơ sở dữ liệu:** SQLite mặc định hoặc kết nối MySQL/PostgreSQL Docker.
- **Quản lý khóa (Crypto Token):** Sử dụng cấu hình SOFT Token mặc định. Hệ thống sinh và bảo vệ khóa mã hóa Root/SubCA trực tiếp bằng thư viện phần mềm tích hợp sẵn của EJBCA thay vì dùng HSM.

Thực nghiệm: Tương tác EJBCA REST API bằng Python

Luồng tự động hóa cấp phát chứng chỉ sử dụng Python Client

Client sử dụng ngôn ngữ Python giao tiếp trực tiếp với cổng EJBCA REST API để tự động sinh khóa và gửi CSR:

Bước 1: Sinh khóa & Tạo CSR trong Python

- Sử dụng thư viện `cryptography.hazmat`.
- Sinh khóa ECC P-256 hoặc RSA 2048-bit.
- Đóng gói DN và ký tạo chuỗi CSR ở dạng PEM.

Bước 2: Giao tiếp REST API qua HTTP POST

- Endpoint: `/ejbca/ejbca-rest-api/v1/certificate/pkcs10enroll`
- Body JSON chứa: `certificate_request` (chuỗi CSR PEM), `username`, `password`, và `certificate_profile_name`.

Bước 3: Tiếp nhận và xử lý của EJBCA

- EJBCA nhận request, xác thực quyền của End Entity qua API.
- Ánh xạ cấu hình Certificate Profile (ràng buộc KeyUsage, EKU, v3 Extensions).
- Ký số và trả về chứng chỉ X.509 PEM cho Client Python.

Bước 4: Sử dụng tại máy khách

- Python nhận Cert, ghi vào file hoặc đóng gói thành PKCS#12 phục vụ kết nối mTLS.

Thực nghiệm CLI và Kết luận

Thực nghiệm qua PKI Manager CLI

Dự án cấu hình công cụ CLI pki trong virtualenv hỗ trợ kết nối API để quản trị CA trực tiếp:

- Tạo CA Root/Intermediate: `pki ca create -cn "My Root CA"`
- Yêu cầu cấp chứng chỉ: `pki cert issue -ca CA_ID -cn "web.local" -type server`
- Thu hồi chứng chỉ: `pki cert revoke CERT_ID -reason keyCompromise`

Kết luận & Xu hướng mới

- PKI là giao thoa giữa toán học mật mã, chính sách vận hành và tự động hóa.
- **Zero Trust:** Mutual TLS (mTLS) bắt buộc xác thực hai chiều cho mọi microservice.
- **Hậu lượng tử (Post-Quantum PQC):** NIST chuẩn hóa thuật toán hình học lưới **ML-KEM** và **ML-DSA** năm 2024. Đòi hỏi PKI hiện đại chuyển dịch sang tính **Crypto-Agility** (Linh hoạt mật mã).

XIN CHÂN THÀNH CẢM ƠN!

PGS. TS. Nguyễn Linh Giang & Hội đồng

HỎI & ĐÁP (Q&A)